

Minor Project

AYNIYH (All You Need Is Your Hand): A C++ Based Hand Gesture Recognition System For Computer Control

Team Members:

Rittik Gan Chaudhuri (23103123)

Manas Raj (23103085)

Vansh Rai (23103159)

Faculty Mentor:

Rahul Aggarwal



Project Proposal & Implementation Workflow

Problem Statement:

CONTEXT: Hand Gesture Recognition (HGR) is a major goal for Human-Computer Interaction (HCI). The vast majority of academic and hobbyist prototypes are built in Python due to its ease of use for AI training.

THE GAP: This reliance on an interpreted language creates a critical "Latency Gap." For HCI to feel 'real-time,' latency must be under ~50ms. Python-based systems, with their interpreter overhead, often exceed 80-120ms.

THE CONSEQUENCE: This gap breaks user immersion and makes the system feel 'sluggish' and 'unusable' as a professional alternative to a mouse. The current prototype-focused approach is unsuitable for production-grade, real-time applications.

THE OBJECTIVE: This project's goal is to solve the Latency Gap by designing, building, and benchmarking a system that moves from a Python prototype to a high-performance C++ deployment, enabling truly instantaneous interaction.

The Problem: The "Latency Gap"

Physical Tethers: We are bound to the mouse and keyboard, which limits accessibility, hygiene for public terminals, and immersive interaction.

The "Lag" Problem: Existing gesture control solutions (mostly Python) are excellent prototypes but suffer from high latency.

Broken Immersion: For Human-Computer Interaction (HCI), a lag of over 50-80ms is noticeable and breaks the user's sense of direct control.

The Need: We need a solution that is not just functional, but instantaneous, with performance on par with a physical device.

Literature Review:

The Status Quo (Python Prototyping): The current, common approach for Hand Gesture Recognition (HGR) *prototypes* heavily relies on Python. It leverages the **MediaPipe** framework for its robust landmark detection and **Keras/TensorFlow** for model training.

- (Ref: *Paudyal, P., et al., 2023; Shrinivas, S. G., et al., 2022*)

Python's Documented Strength (Rapid Iteration): Python's primary, documented strength is in **rapid prototyping** and interactive scientific computing. Its ecosystem (NumPy, Keras, etc.) makes it the undisputed standard for *training* and *iterating* on complex AI models.

- (Ref: *Perez, F. & Granger, B. E., 2007; Oliphant, T. E., 2007*)

The Flaw (The "Latency Gap"): This reliance on Python for *deployment* creates a critical "**Latency Gap.**" Python's interpreted nature and the Global Interpreter Lock (GIL) introduce significant overhead. Direct, peer-reviewed benchmarks show C++ implementations of the *exact same* OpenCV tasks performing between **5.25x and 15x faster** than their Python-wrapper equivalents.

- (Ref: *G. P. R, A. S., et al., 2022; S. R. S, S. V., 2020*)

Literature Review:

The SOTA Goal (Efficiency): State-of-the-art (SOTA) research in Human-Computer Interaction (HCI) is defined not just by accuracy, but by **low latency** and **computational efficiency**. A seamless, real-time experience requires minimizing "computational time" to be truly interactive.

- (Ref: *Rautaray, S. S. & Agrawal, A., 2015*)

Conclusion (Our Project's Justification): A Python-only approach is sufficient for *prototyping* but fails to meet SOTA performance goals for a *real-time* system. Therefore, our project's hybrid workflow—training in Python, deploying in C++—directly addresses this documented "Latency Gap" and aligns our work with industry-standard best practices.

The Proposed Solution: AYNIIYH

A hybrid, high-performance C++ application that gets the best of both worlds:

- **The "Lab" (Python):** Use Python only for what it's good at: rapidly training a flexible 1D Convolutional Neural Network (CNN) model.
- **The "Factory" (C++):** Deploy the trained model (as an `.onnx` file) in a lightweight, native C++ application for maximum speed and minimal latency.

Proposed Tech Stack



Python (The Lab)

Use Keras/TensorFlow to design and train our 1D CNN gesture classification model.



ONNX (The Blueprint)

Export the trained model to ONNX, a format that bridges the gap between Python and C++.



C++ (The Factory)

The core language for the final, high-speed application that the user will run.



OpenCV (The Engine)

Used in C++ for two things: capturing the webcam feed and running the ONNX model



Robot-CP P (The Hands)

A pre-built C++ library for cross-platform OS control (mouse, keyboard).

Workflow: Phase 1 - The "Brain" (Python)

This phase is done once, in our "lab."

1. Use MediaPipe to capture 3D "World Landmarks" (a 63-point vector, 21x3).
2. Collect 100s of samples for each gesture (`IDLE`, `POINT`, `CLICK`, `FIST`).
3. Train a 1D CNN to classify these vector sequences.
4. Export the final, optimized model to `gesture\_cnn.onnx`.

Workflow: Phase 2 - The "Body" (C++)

We will build the C++ application using a clean, modular design.

 **`HandTracker.cpp` (The Eyes):** Runs the MediaPipe TFLite models to get the 63-point landmark vector from the webcam.

 **`GestureCNN.cpp` (The Brain):** Loads `gesture\_cnn.onnx` and has one job: predict a gesture ID from the landmark vector.

 **`Robot-CPP` (The Hands):** A pre-built library for all OS-specific commands.

 **`main.cpp` (The Conductor):** Connects all modules in a clean, high-speed loop.

The C++ Advantage: `robot-cpp`

By using the `robot-cpp` library, we avoid writing thousands of lines of platform-specific code (`windows.h`, `X11`).

This keeps our `main.cpp` logic 100% clean and portable. We simply call its high-level, readable functions:

```
Robot::Mouse::Move(x, y);  
Robot::Mouse::Click();  
Robot::Keyboard::Press(Key::Space);
```

Proposed Project Timeline (4 Weeks)

Week 2: Core

Build & Test

``GestureCNN.cpp`` (AI Inference) and integrate ``robot-cpp``.

Week 4: Report

Testing, bug-fixing, and performance benchmarking (C++ vs. Python).

Week 1: Setup

Set up C++ Environment (vcpkg, OpenCV, robot-cpp). Train V1 Gesture Model in Python.

Week 3: Integration

Build ``HandTracker.cpp`` (Vision) & integrate all modules in ``main.cpp``.

Expected Outcomes

<20ms

Target End-to-End Latency

Project Deliverables

- A functional C++ application for controlling the mouse, clicks, and keyboard via gestures.
- A scalable system where new gestures can be added just by retraining the `.onnx` model.
- A final report benchmarking our C++ latency vs. a baseline Python equivalent.

Challenges & Mitigation

Risk 1: C++ Environment

Risk: Linking C++ libraries (OpenCV, TFLite) is notoriously complex and time-consuming.

Mitigation: Use `vcpkg` as a package manager from Day 1. This will be the entire focus of Week 1.

Risk 2: Model Accuracy

Risk: The 1D CNN may confuse similar gestures, like "Point" (Move) and "Pinch" (Click).

Mitigation: Collect a large, diverse training dataset. The model is only as good as its data.

References

- G. P. R, A. S., et al. (2022). "Performance Analysis of Python and C++ in OpenCV." *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 10(6).
- Oliphant, T. E. (2007). "Python for Scientific Computing." *Computing in Science & Engineering*, 9(3).
- Paudyal, P., et al. (2023). "Real-time Hand Gesture Recognition using MediaPipe and TensorFlow." *In 2023 5th International Conference on ICT & Operations (ICT-OP)*.
- Perez, F., & Granger, B. E. (2007). "IPython: A System for Interactive Scientific Computing." *Computing in Science & Engineering*, 9(3).
- Rautaray, S. S., & Agrawal, A. (2015). "Vision based hand gesture recognition for human computer interaction: a survey." *Artificial Intelligence Review*, 43(1).
- S. R. S, S. V. (2020). "A Comparative Study of Python and C++ Performance in OpenCV." *International Journal of Research in Engineering, Science and Management*, 3(6).
- Shrinivas, S. G., et al. (2022). "A Real-Time Hand Gesture Recognition with MediaPipe." *In 2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*.

Thank You